

Task1

```
package org.example;

import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.*;
import java.net.URI;

//create table in DynamoDB
public class CreateTable {
    public static void main(String[] args) throws Exception {
        System.out.println("hello create table in DynamoDB");
        AwsBasicCredentials awsCreds = AwsBasicCredentials.create("accessKeyId: "fakeAccessKey");

        DynamoDbClient client = DynamoDbClient.builder()
            .endpointOverride(URI.create("http://localhost:8007"))
            .region(Region.AP_SOUTH_1)
            .credentialsProvider(StaticCredentialsProvider.create(awsCreds))
            .build();

        String tableName = "Harika_Table";

        try {
            CreateTableRequest request = CreateTableRequest.builder()
                .tableName(tableName)
                .build();
        }
    }
}
```

```

try {
    CreateTableRequest request = CreateTableRequest.builder()
        .tableName(tableName)
        .keySchema(KeySchemaElement.builder()
            .attributeName(s: "ID")
            .keyType(KeyType.HASH)
            .build())
        .attributeDefinitions(AttributeDefinition.builder()
            .attributeName(s: "ID")
            .attributeType(ScalarAttributeType.N)
            .build())
        .provisionedThroughput(ProvisionedThroughput.builder()
            .readCapacityUnits(aLong: 5L)
            .writeCapacityUnits(aLong: 5L)
            .build())
        .build();
    client.createTable(request);
    System.out.println(tableName + " table is created. ");
} catch (Exception ex) { //(ResourceInUseException ex) {
    System.out.println(" plz choose different tablename as it already exists");
}
client.close();

```

```

"D:\Program Files\Java\jdk-24\bin\java.exe" ...
hello create table in DynamoDB
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Harika_Table table is created.

Process finished with exit code 0

```

Task2

```
package org.example;
import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.*;
import java.net.URI;

//create table in DynamoDB
public class CreateTable {
    public static void main(String[] args) throws Exception {
        System.out.println("hello create table in DynamoDB");
        AwsBasicCredentials awsCreds = AwsBasicCredentials.create("accessKeyId: "fakeAccesske

        DynamoDbClient client = DynamoDbClient.builder()
            .endpointOverride(URI.create("http://localhost:8000"))
            .region(Region.AP_SOUTH_1)
            .credentialsProvider(StaticCredentialsProvider.create(awsCreds))
            .build();

        String tableName = "Kantipudi_Table";

        try {
            CreateTableRequest request = CreateTableRequest.builder()
                .tableName(tableName)
                .keySchema(KeySchema.builder()
                    .addKey(KeyType.PARTITION, "id")
                    .build())
                .attributeDefinitions(AttributeDefinition.builder()
                    .attributeName("id")
                    .attributeDataType(ScalarAttributeDataType.STRING)
                    .build())
                .build();

            client.createTable(request);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

        CreateTableRequest request = CreateTableRequest.builder()
            .tableName(tableName)
            .keySchema(KeySchemaElement.builder()
                .attributeName(s: "ID")
                .keyType(KeyType.HASH)
                .build())
            .attributeDefinitions(AttributeDefinition.builder()
                .attributeName(s: "ID")
                .attributeType(ScalarAttributeType.N)
                .build())
            .provisionedThroughput(ProvisionedThroughput.builder()
                .readCapacityUnits(aLong: 5L)
                .writeCapacityUnits(aLong: 5L)
                .build())
            .build();
        client.createTable(request);
        System.out.println(tableName + " table is created. ");
    } catch (Exception ex) { //(ResourceInUseException ex) {
        System.out.println(" plz choose different tablename as it already exists");
    }
    client.close();
}

```

```

"D:\Program Files\Java\jdk-24\bin\java.exe" ...
hello create table in DynamoDB
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Kantipudi_Table table is created.
|
Process finished with exit code 0

```

Task4

```

package org.example;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.AttributeValue;
import software.amazon.awssdk.services.dynamodb.model.PutItemRequest;

import java.io.InputStream;
import java.net.URI;
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;

// loading data to our DynamoDB table
public class LoadingData02 {
    public static void main(String[] args) throws Exception {
        // using theb Aws credentials
        AwsBasicCredentials awsCreds = AwsBasicCredentials.create("accessKeyId: "fakeaccess", se

        // create a DynamoDb client

```

```

        // create a DynamoDb client
        DynamoDbClient client = DynamoDbClient.builder()
            .endpointOverride(URI.create("http://localhost:8000"))
            .region(Region.AP_SOUTH_1)
            .credentialsProvider(StaticCredentialsProvider.create(awsCreds))
            .build();
        System.out.println("connection created successfully");
        String tableName = "Employees04";

        //object Mapper
        ObjectMapper mapper = new ObjectMapper();

        //giving input stream of data
        InputStream stream = LoadingData02.class.getClassLoader()
            .getResourceAsStream("Employee.json");
        System.out.println("the json file in the input stream");
        JsonNode node = mapper.readTree(stream);

        Iterator<JsonNode> iterator = node.elements();

```

```

while (iterator.hasNext()) {
    JsonNode Jsonnode2 = iterator.next();
    Map<String, AttributeValue> item = new HashMap<>();
    item.put("ID", AttributeValue.builder().n(Jsonnode2.get("ID").asText()).build());
    item.put("Name", AttributeValue.builder().s(Jsonnode2.get("Name").asText()).build());
    item.put("Address", AttributeValue.builder().s(Jsonnode2.get("Address").asText()).build());

    PutItemRequest request = PutItemRequest.builder()
        .tableName(tableName)
        .item(item)
        .build();

    client.putItem(request);
    System.out.println(" loading the data to the table");
}
client.close();
System.out.println("closing client connection");

```

Task5

```

package org.example;
import software.amazon.awssdk.auth.credentials.AwsBasicCredentials;
import software.amazon.awssdk.auth.credentials.StaticCredentialsProvider;
import software.amazon.awssdk.regions.Region;
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.ScanRequest;
import software.amazon.awssdk.services.dynamodb.model.ScanResponse;

import java.net.URI;

public class ScanTable {

    public static void main(String[] args) throws Exception {

        // 1. Connect to DynamoDB Local
        DynamoDbClient client = DynamoDbClient.builder()
            .endpointOverride(new URI(str: "http://localhost:8000"))
            .credentialsProvider(StaticCredentialsProvider.create(
                AwsBasicCredentials.create( accessKeyId: "fakeaccess", secretAccessKey: "
            ))
            .region(Region.AP_SOUTH_1)
            .build();

        System.out.println("✅ Connected to DynamoDB Local");
    }
}

```

```

System.out.println("✅ Connected to DynamoDB Local");

// 2. Scan table Employees04
ScanRequest scanRequest = ScanRequest.builder()
    .tableName(s: "Employees04")
    .build();

ScanResponse response = client.scan(scanRequest);

// 3. Print items
System.out.println("\n📁 Employees in Table:");
response.items().forEach( Map<String, AttributeValue> item -> {
    System.out.println(
        "ID=" + item.get("ID").n() +
        ", Name=" + item.get("Name").s() +
        ", Address=" + item.get("Address").s()
    );
});

client.close();

```

✅ Connected to DynamoDB Local

📁 Employees in Table:

ID=1003, Name=K, Address=USA

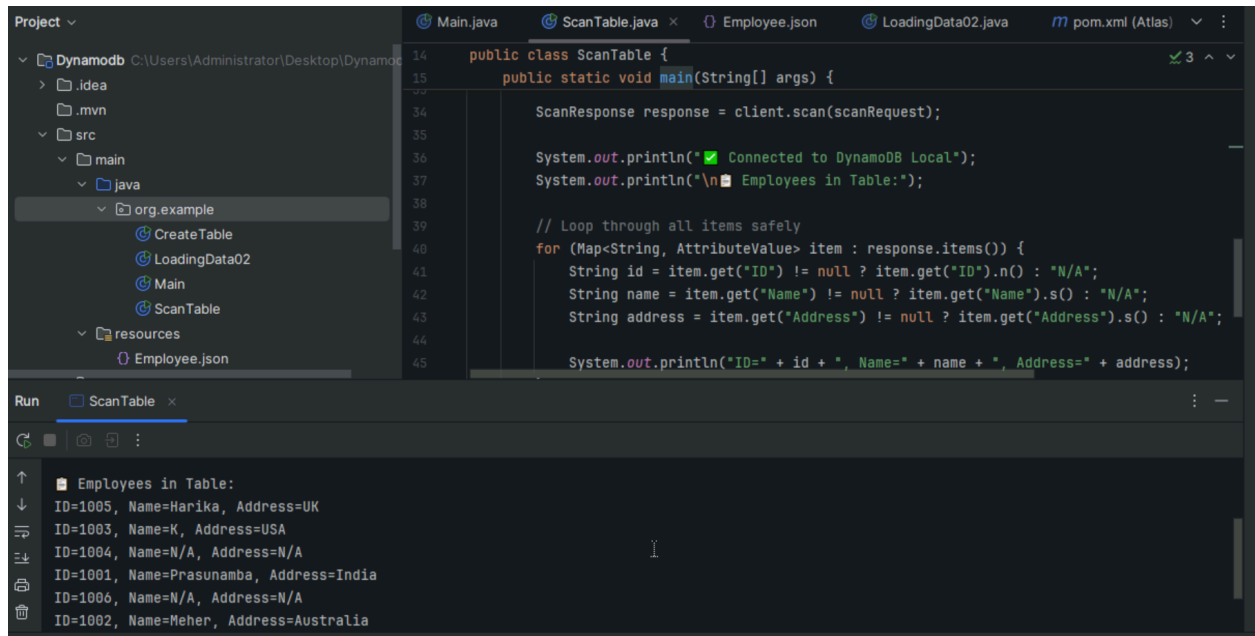
ID=1001, Name=Prasunamba, Address=India

ID=1002, Name=Meher, Address=Australia

✅ Scan complete

Process finished with exit code 0

Task6



Task7